JNEOPALLIUM · INDUSTRIAL DEMOS

Two Models, One Runtime — Live

Supervised loop guardianship and label-free predictive maintenance, demonstrated end to end

Technical walkthrough · Deterministic · Offline · Advisory-only · On-premises

01 What you're about to watch

Two industrial models on the same Jneopallium runtime, with the same safety posture.

- **Industrial Loop Guardian** — supervised diagnosis + advisory optimisation above PLC/PID/SIS.
- **Self-Supervised Maintenance Guardian** — label-free degradation detection that learns from operator feedback.
- Both are **advisory-only**; the hard interlock stays deterministic and outside the model.
- Everything is deterministic, runs offline, and touches no device — safe to run live.

02 Which model, when

Loop Guardian — you have labels

- A labelled failure / maintenance history
- Tuned PID / cascade loops to optimise
- Supervised diagnosis + setpoint advice
- FMI thermal-skid demo, 9 scenarios

Self-Supervised — you don't

- Telemetry but no clean failure log
- Learns 'normal' from the data itself
- Names drift + lead time, no labels
- Improves from one operator click

03 Demo A — Loop Guardian: what you'll see

Run: python run_demo.py all (9 deterministic scenarios).

- **pump-wear** — a developing fault flagged early, with zero false positives.
- **high-temperature-interlock** — the deterministic safety layer trips, not the model.
- **mqtt / opcua outage** — advisory path degrades gracefully; control is unaffected.
- Every scenario writes a reproducible trace + manifest under target/.

04 Loop Guardian — the numbers (real)

From python run_demo.py all — deterministic.

58.4 s	0	0.1 s	0.0
Pump-wear fault detection delay	False positives across 9 scenarios	Interlock response (deterministic, not AI)	Time outside safety bounds (non-interlock)

05 Demo B — Self-Supervised: what you'll see

Run: run_demo.ps1 — the REAL Java neurons replaying a telemetry stream.

- A bearing-wear fault on PUMP-102 flagged **~730 ticks before** the modelled failure — no labels.
- Benign excursions on healthy PUMP-101 flagged as nuisances.
- Operator marks them **false-positive**; the threshold moves **live**, no redeploy.
- Later nuisances are **suppressed**, while the real fault keeps firing above the raised bar.

06 Self-Supervised — the numbers (real)

From the live demo transcript + the test suites.

0	7	1.00 → 1.37	14 + 5
Failure labels used in training	Nuisance flags suppressed by feedback	Bearing threshold learned live	Java + Python tests passing

07 The safety boundary — say it once, clearly

The only thing that 'acted' in the whole demo was the deterministic interlock.

- Both models **only advise** — advisory-only is an invariant in the code, not a policy.
- The **hard interlock** responded in 0.1 s — the AI never actuates.
- Self-supervised learning touches **thresholds**, never a safety gate or interlock.
- Rollout is **shadow → advisory**; there is no autonomous stage.

08 What the demo proves — and what it doesn't

It proves

- The runtime logic is correct (tested)
- Faults separate with zero labels
- Feedback cuts nuisances, not real faults
- The model cannot actuate

It doesn't claim

- A field accuracy rate from a bench run
- Certainty of a family pre-confirmation
- Zero false positives on real plants
- Value without a healthy baseline

Same runtime. Same safety. Two entry points.

Have labels? Loop Guardian. Don't? Self-Supervised — and it learns from there.

Next step: a shadow-mode pilot on your telemetry, on your premises.

Book a shadow pilot